

Avaliacao de desempenho do Crivo de Eratostenes com Perf

Programa avaliado: Crivo de Eratostenes para `n = 100000000`, comparando uma versao sequencial e uma versao paralela OpenMP limitada a dois nucleos/threads.

Observacao: os valores da tabela abaixo sao simulados/ilustrativos, pois esta maquina esta em Windows, sem `gcc`, sem `perf` e sem uma distribuicao WSL pronta para executar a coleta real. A pasta `ativ10` contem as fontes e o script `run_perf.sh` para coleta direta em Linux.

Comandos de coleta

```
cd ativ10
gcc -O3 -march=native crivo_seq.c -lm -o crivo_seq
gcc -O3 -march=native -fopenmp crivo_omp2.c -lm -o crivo_omp2

perf stat -r 3 -e task-clock,cycles,instructions,stalled-cycles-frontend,stalled-cycles-backend,LLC-loads,LLC-load-misses ./crivo_seq 100000000
OMP_NUM_THREADS=2 perf stat -r 3 -e task-clock,cycles,instructions,stalled-cycles-frontend,stalled-cycles-backend,LLC-loads,LLC-load-misses ./crivo_omp2 100000000
```

Tabela comparativa

Metrica do Perf	Sequencial	Paralelo - 2 nucleos	Interpretacao esperada
CPUs utilized	0,99	1,78	A versao paralela usa mais de um nucleo, mas nao chega a 2,00 por causa de trechos seriais, barreiras OpenMP e espera por memoria.
Frontend cycles idle	4,1%	7,8%	A versao paralela tem maior custo de controle e sincronizacao, aumentando ciclos em que o processador nao recebe instrucoes eficientemente.
Backend cycles idle	58,2%	72,4%	O backend fica mais ocioso na versao paralela, indicando espera por memoria/cache devido ao acesso compartilhado e irregular ao vetor <code>prime[]</code> .

Mettrica do Perf	Sequencial	Paralelo - 2 nucleos	Interpretacao esperada
Instructions per cycle (IPC)	0,62	0,41	O IPC menor no paralelo mostra que adicionar threads nao aumentou proporcionalmente o trabalho util por ciclo.
LL-cache hits / taxa de falta L3	18,42 M loads; 2,61 M misses; miss rate 14,2%	31,85 M loads; 6,72 M misses; miss rate 21,1%	A taxa de falta L3 maior no paralelo reforca que a aplicacao esta limitada por memoria e compartilhamento de cache.
Time elapsed	9,84 s	6,37 s	O speedup aproximado e 1,54x, abaixo do ideal de 2x, coerente com os gargalos de sincronizacao e memoria.

Gargalos da versao paralela

- **Sincronizacao frequente:** o `#pragma omp for` esta dentro do loop de `p`. Cada primo encontrado introduz uma divisao de trabalho e uma barreira implicita, aumentando o custo de coordenacao.
- **Dependencia no vetor compartilhado:** todas as threads leem e escrevem em `prime[]`. Mesmo que escrever `false` repetidamente nao altere o resultado, isso gera trafego de cache e possivel invalidacao entre nucleos.
- **Acesso a memoria pouco local:** para cada `p`, o loop marca posicoes com salto `p`. Isso reduz localidade espacial, aumenta pressoes na hierarquia de cache e pode elevar ciclos ociosos no backend.
- **Trabalho redundante:** a marcacao comeca em `2 * p`. Numeros menores que `p * p` ja foram marcados por primos anteriores.
- **Baixo paralelismo efetivo no inicio:** para valores pequenos de `p`, ha muito trabalho, mas muitas escritas concorrentes no mesmo vetor; para valores maiores, ha menos iteracoes e a distribuicao entre duas threads fica menos eficiente.

Otimizacao proposta

Uma melhoria direta e iniciar a marcacao em `p * p`, em vez de `2 * p`. Isso elimina marcacoes redundantes ja realizadas por primos menores, reduz a quantidade de instrucoes executadas e diminui trafego de memoria/cache.

Uma otimizacao mais forte seria usar um crivo segmentado: cada thread processa blocos independentes do intervalo, usando a lista de primos ate `sqrt(n)`. Essa abordagem melhora localidade de cache, reduz disputa no vetor global e diminui invalidacoes entre nucleos.

Conclusao

O principal limite esperado para a versao paralela nao e falta de nucleos, mas custo de memoria e sincronizacao. A paralelizacao atual divide a marcacao para cada primo, porem paga barreiras frequentes e compartilha intensamente o vetor `prime[]`. Por isso, mesmo com dois nucleos, o ganho tende a ficar abaixo de 2x.